Wheels Alpes – Entrega #2

1. **<u>Análisis y revisión del modelo de datos.</u>**

En el análisis de la retroalimentación de la Entrega #1 no se pidió cambiar entidades, relaciones ni cardinalidades. Por lo tanto, no se crearon tablas nuevas ni se eliminaron tablas: el modelo conserva las 12 relaciones de la Entrega 1. Los ajustes se concentran en las restricciones de integridad (dominio y unicidad) y en la coherencia entre el modelo documentado y la base de datos implementada.
A continuación, se presentan los siguientes cambios sugeridos en la retroalimentación:

Clase	**Cambios**
Usuario	CK(Documento_ID > 0), CK de correo institucional (LIKE '%@%.edu.co'), CK de celular (10 dígitos y empieza en 3), UK sobre Correo
Carros	CK(LENGTH(Placa) = 6), CK(Ano BETWEEN 1990 AND 2030), CK(Cupos BETWEEN 1 AND 6)
Viajes	CK(Costo > 0), CK(Cupos_disponibles >= 0), CK(Distancia_estimada > 0), CK(Duracion_estimada > 0)
Paradas_viaje	CK(Numero_parada > 0), UK(Viajes_ID_viaje, Numero_parada), de modo que un viaje no repite el orden de sus paradas.
Estados_viaje	UK(Nombre_estado), CK(Nombre_estado IN Programado, En curso, Completado, Cancelado).
Estados_solicitud	UK(Nombre_solicitud), CK(Nombre_solicitud IN Pendiente, Aceptada, Rechazada, Cancelada).

<u>Datos:</u>

- Las demás entidades conservan su antigua información, puesto que no había necesidad de cambios.
- El modelo relacional se alineó con el script de creación: los nombres de atributos quedaron sin tildes (Ano, Direccion, Duracion_estimada, Numero_celular, Numero_parada, Descripcion), los tipos de fecha son TIMESTAMP y las llaves foráneas hacia Viajes usan el mismo tipo que su llave primaria (NUMERIC).
- Las entidades Estados_viaje y Estados_solicitud son los catálogos de estados. Estado_viaje y Estados_de_solicitud son las tablas intermedias (relaciones N:M) que guardan el historial de estados con su fecha. Por eso existen cuatro tablas con nombres parecidos y ninguna es redundante.
- Se usa una sola tabla Usuario que refiere a conductores y pasajeros. Un usuario se considera conductor cuando tiene al menos un vehículo registrado en Carros, pero un usuario se considera pasajero cuando se inscribe para pedir un viaje. Esta regla es la que usa la aplicación para decidir el esquema de usuario autenticado.

- Anteriormente mencionado en la Entrega #1, todas las entidades propuestas cumplen con la verificación de la forma normal de Boyce-Codd (BCNF): Las 12 relaciones cumplen 1FN (atributos atómicos), 2FN (ningún atributo depende de una parte de una llave compuesta), 3FN (no hay dependencias transitivas) y BCNF (todo determinante es llave candidata). Por lo tanto, no fue necesario descomponer ninguna tabla y el modelo final es el mismo de la Entrega #1, con las restricciones completadas.

2. Modelo relacional.

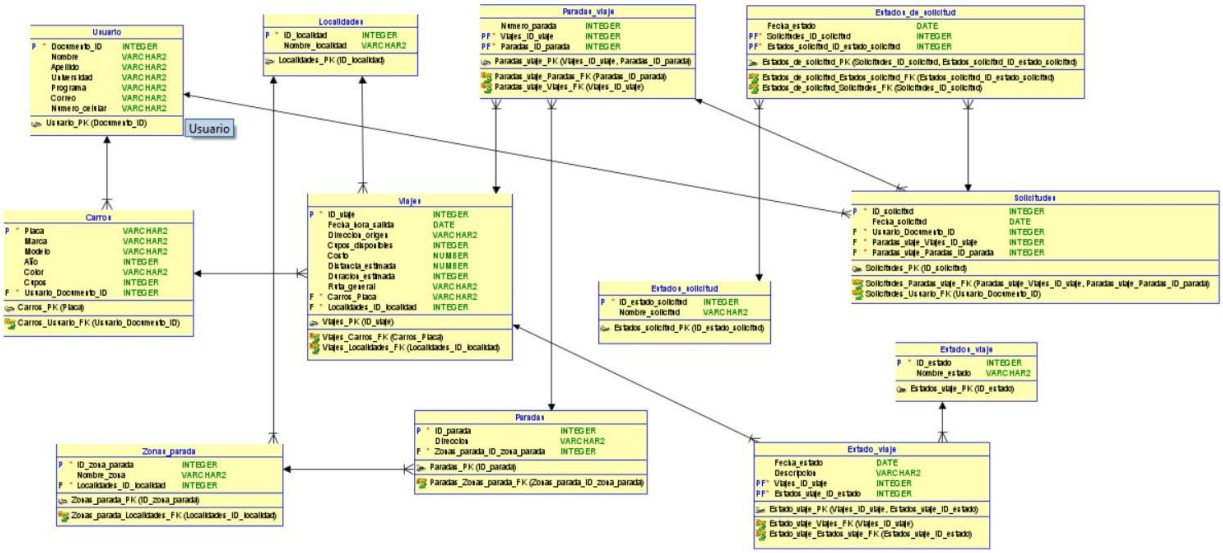


Ilustración 1. Modelo relacional de Wheels Alpes (Oracle Data Modeler).

Con la información del modelo relacional actualizada, se presenta el análisis correspondiente a cada entidad:

Entidad	Atributos	Restricciones
Localidades	ID_localidad (INTEGER), Nombre_localidad (VARCHAR 255)	PK(ID_localidad), NN en todos
Zonas_parada	ID_zona_parada (INTEGER), Nombre_zona (VARCHAR 255), Localidades_ID_localidad (INTEGER)	PK(ID_zona_parada), FK(Localidades_ID_localidad) → Localidades, NN en todos
Paradas	ID_parada (INTEGER), Direccion (VARCHAR 255), Zonas_parada_ID_zona_parada (INTEGER)	PK(ID_parada), FK(Zonas_parada_ID_zona_parada) → Zonas_parada, NN en todos
Usuario	Documento_ID (INTEGER), Nombre, Apellido, Universidad, Programa, Correo, Numero_celular (VARCHAR 255)	PK(Documento_ID), UK(Correo), NN en todos, CK documento, correo y celular
Carros	Placa (VARCHAR 20),	PK(Placa),

	Marca, Modelo, Color (VARCHAR 255), Ano, Cupos (INTEGER), Usuario_documento_ID (INTEGER)	FK(Usuario_documento_ID) → Usuario, NN en todos, CK placa, ano y cupos
Viajes	ID_viaje (NUMERIC), Fecha_hora_salida (TIMESTAMP), Direccion_origen (VARCHAR 255), Cupos_disponibles, Costo, Distancia_estimada, Duracion_estimada (NUMERIC), Ruta_general (VARCHAR 255), Carros_placa (VARCHAR 20), Localidades_ID_localidad (INTEGER)	PK(ID_viaje), FK(Carros_placa) → Carros, FK(Localidades_ID_localidad) → Localidades, NN en todos, CK cupos, costo, distancia y duración
Paradas_viaje	Viajes_ID_viaje (NUMERIC), Paradas_ID_parada (INTEGER), Numero_parada (INTEGER)	PK(Viajes_ID_viaje, Paradas_ID_parada), FK a Viajes y a Paradas, UK(Viajes_ID_viaje, Numero_parada), CK(Numero_parada > 0)
Estados_viaje	ID_estado (INTEGER), Nombre_estado (VARCHAR 255)	PK(ID_estado), UK(Nombre_estado), CK(Nombre_estado)
Estado_viaje	Viajes_ID_viaje (NUMERIC), Estados_viaje_ID_estado (INTEGER), Fecha_estado (TIMESTAMP), Descripcion (VARCHAR 255)	PK(Viajes_ID_viaje, Estados_viaje_ID_estado), FK a Viajes y a Estados_viaje
Solicitudes	ID_solicitud (INTEGER), Fecha_solicitud (TIMESTAMP), Usuario_Documento_ID (INTEGER), Paradas_viaje_Viajes_ID_viaje (NUMERIC), Paradas_viaje_Paradas_ID_parada (INTEGER)	PK(ID_solicitud), FK(Usuario_Documento_ID) → Usuario, FK compuesta (viaje, parada) → Paradas_viaje, NN en todos
Estados_solicitud	ID_estado_solicitud (INTEGER), Nombre_solicitud (VARCHAR 255)	PK(ID_estado_solicitud), UK(Nombre_solicitud), CK(Nombre_solicitud)
Estados_de_solicitud	Solicitudes_ID_solicitud (INTEGER), Estados_solicitud_ID_estado_solicitud (INTEGER), Fecha_estado (TIMESTAMP)	PK(Solicitudes_ID_solicitud, Estados_solicitud_ID_estado_solicitud), FK a Solicitudes y a Estados_solicitud

3. Reglas de negocio.

Las siguientes mencionadas no hacen parte de las restricciones en los scripts desarrollados. Sin embargo, las implementamos en la lógica de la entrega para aclaración de las validaciones y procesos de la página web.

- Los cupos disponibles del viaje no pueden superar los cupos establecidos.
- El usuario (conductor) no puede solicitar cupo en su propio viaje.
- Para aceptar viajes solicitados, solo está permitido cuando se encuentra en el estado “Programado”.
- Referente a la secuencia de viajes, cuando su estado sea “Completado”, significa que el viaje no se puede modificar.

- Como se estableció en los requerimientos funcionales, no se puede cancelar un viaje o una solicitud, se debe informar con al menos dos horas de anticipación respecto a la hora de salida del viaje.

4. Implementación de la base de datos.

- Herramientas: la entrega fue desarrollada en Oracle APEX, usando SQL mediante archivos de script ejecutados desde SQL Scripts.
- Scripts implementados para la entrega:

Archivo	Contenido
01_ddl_tablas	Contiene las 12 tablas con sus PK, FK, UK, NOT NULL y los CHECK.
02_datos_estados	Contiene los 4 estados de viaje (Programado, En curso, Completado, Cancelado) y los 4 estados de solicitud (Pendiente, Aceptada, Rechazada, Cancelada).
03_checks	Contiene los CHECK necesarios.
04_poblacion	
05_credenciales	
06_triggers	

5. Documentación RFC y RFA.

Decisiones generales:

- En el modelo Viajes ni Solicitudes tienen una columna de estado. El estado se saca de las tablas de historial (Estado_viaje y Estados_de_solicitud), que guardan una fila por cada estado por el que pasó el viaje o la solicitud. Como la PK de esas tablas es la combinación de la entidad y el estado, cada estado puede aparecer máximo una vez. Los estados se buscan por su nombre en los catálogos (Estados_viaje y Estados_solicitud) y no por su ID, para que las consultas sigan funcionando, aunque los ID's del catálogo cambien.

- Para revisar el historial se usa EXISTS y NOT EXISTS en vez de un INNER JOIN. Si se hace un INNER JOIN con el historial, cada viaje o solicitud se repite una vez por cada estado que tenga, y eso daña los conteos y las sumas. Con EXISTS solo se pregunta si existe una fila con ese estado, sin agregar filas al resultado.

- El atributo Localidades_ID_localidad de Viajes se toma como la localidad de origen del viaje, es decir, la localidad desde donde sale el conductor. Cuando lo que importa es la demanda de un pasajero, se usa en cambio la localidad de su punto de recogida, a la que se llega por Paradas → Zonas_parada → Localidades.

- Las fechas de solicitudes transcurren entre 22-Oct-26 al 09-Nov-26.

RFC1 - Consultar demanda por localidad.

Para esta consulta se interpretó la demanda como la cantidad de solicitudes de cupo que se hicieron en un rango de fechas, agrupadas por la localidad del punto de recogida que eligió el pasajero. Se cuentan todas las solicitudes sin importar su estado, ya que una solicitud rechazada también es demanda.

C: > Users > david > Downloads > rfc1_demanda_por_localidad.sql

```sql
SELECT l.Nombre_localidad,
       COUNT(s.ID_solicitud) AS total_solicitudes
FROM Solicitudes s
INNER JOIN Paradas p      ON p.ID_parada      = s.Paradas_viaje_Paradas_ID_parada
INNER JOIN Zonas_parada z ON z.ID_zona_parada = p.Zonas_parada_ID_zona_parada
INNER JOIN Localidades l  ON l.ID_localidad   = z.Localidades_ID_localidad
WHERE s.Fecha_solicitud >= TO_DATE(:P_FECHA_INICIO, 'YYYY-MM-DD')
  AND s.Fecha_solicitud <  TO_DATE(:P_FECHA_FIN, 'YYYY-MM-DD') + 1
GROUP BY l.ID_localidad, l.Nombre_localidad
ORDER BY total_solicitudes DESC;
```

Tabla	Para qué se usa	Atributos del JOIN
Solicitudes	Cada fila es una solicitud. De aquí sale la fecha y el punto de recogida	Paradas_viaje_Paradas_ID_parada
Paradas	Ubica el punto de recogida en una zona	ID_parada, Zonas_parada_ID_zona_parada
Zonas_parada	Conecta la zona con su localidad	ID_zona_parada, Localidades_ID_localidad
Localidades	Da el nombre por el que se agrupa	ID_localidad

Se usó INNER JOIN en todos los casos porque cada solicitud tiene un punto de recogida, y tanto las paradas como las zonas tienen sus FK obligatorias, entonces ningún JOIN deja por fuera solicitudes que sí deberían contarse. Además, con INNER JOIN solo aparecen las localidades que tuvieron al menos una solicitud en el período, que es lo que se quiere mostrar.

No se usó la tabla Paradas_viaje porque Solicitudes ya tiene el ID de la parada dentro de su FK compuesta, así que unirla no aportaba ninguna columna nueva.

RFC2 - Consultar ingresos por localidad.

Para esta consulta se tomó el ingreso estimado de una localidad como la suma del costo de cada cupo vendido en los viajes que comenzaron desde esa localidad, cuya fecha de salida esté dentro del rango. Un cupo vendido es una solicitud que fue aceptada, que no se canceló después y cuyo viaje tampoco fue cancelado. Como todos los pasajeros de un viaje pagan el mismo precio, sumar el Costo una vez por cada solicitud da lo mismo que multiplicar el costo por la cantidad de pasajeros del viaje.

```sql
SELECT l.Nombre_localidad,
       COUNT(*)     AS cupos_vendidos,
       SUM(v.Costo) AS ingresos_estimados
FROM Solicitudes s
INNER JOIN Viajes v      ON v.ID_viaje     = s.Paradas_viaje_Viajes_ID_viaje
INNER JOIN Localidades l ON l.ID_localidad = v.Localidades_ID_localidad
WHERE v.Fecha_hora_salida >= TO_DATE(:P_FECHA_INICIO, 'YYYY-MM-DD')
  AND v.Fecha_hora_salida <  TO_DATE(:P_FECHA_FIN, 'YYYY-MM-DD') + 1
  -- la solicitud fue aceptada...
  AND EXISTS (SELECT 1
              FROM Estados_de_solicitud e
              INNER JOIN Estados_solicitud es
                      ON es.ID_estado_solicitud = e.Estados_solicitud_ID_estado_solicitud
              WHERE e.Solicitudes_ID_solicitud = s.ID_solicitud
                AND es.Nombre_solicitud = 'Aceptada')
  -- ...y no se canceló después
  AND NOT EXISTS (SELECT 1
                  FROM Estados_de_solicitud e
                  INNER JOIN Estados_solicitud es
                          ON es.ID_estado_solicitud = e.Estados_solicitud_ID_estado_solicitud
                  WHERE e.Solicitudes_ID_solicitud = s.ID_solicitud
                    AND es.Nombre_solicitud = 'Cancelada')
  -- el viaje no fue cancelado
  AND NOT EXISTS (SELECT 1
                  FROM Estado_viaje ev
                  INNER JOIN Estados_viaje esv
                          ON esv.ID_estado = ev.Estados_viaje_ID_estado
                  WHERE ev.Viajes_ID_viaje = v.ID_viaje
                    AND esv.Nombre_estado = 'Cancelado')
GROUP BY l.ID_localidad, l.Nombre_localidad
ORDER BY ingresos_estimados DESC;
```

Tabla	Para qué se usa	Atributos del JOIN
Solicitudes	Cada fila es un cupo solicitado	Paradas_viaje_Viajes_ID_viaje, ID_solicitud
Viajes	Da el costo, la fecha de salida y la localidad	ID_viaje, Localidades_ID_localidad
Localidades	Da el nombre por el que se agrupa	ID_localidad
Estados_de_solicitud y Estados_solicitud	Historial de la solicitud, dentro de EXISTS y NOT EXISTS	Solicitudes_ID_solicitud, Estados_solicitud_ID_estado_solicitud, ID_estado_solicitud
Estado_viaje y Estados_viaje	Historial del viaje, dentro de NOT EXISTS	Viajes_ID_viaje, Estados_viaje_ID_estado, ID_estado

Con Viajes y Localidades se usó INNER JOIN porque toda solicitud pertenece a un viaje y todo viaje tiene una localidad, ya que las FK son obligatorias. Así no se pierde ninguna fila que sí deba contarse, y solo aparecen las localidades que generaron ingresos.

Con los historiales se usó EXISTS y NOT EXISTS. Una solicitud puede tener varias filas en su historial (Pendiente, Aceptada y a veces Cancelada), entonces si se hiciera un INNER JOIN la solicitud se repetiría una vez por cada estado y su costo se sumaría varias veces. Con EXISTS se revisa que haya tenido el estado 'Aceptada' y con NOT EXISTS que nunca haya tenido 'Cancelada', sin duplicar filas. Lo mismo pasa con el historial del viaje.

Aquí se usó la localidad de origen del viaje y no la del punto de recogida porque el ingreso lo genera el viaje, y el costo es un atributo del viaje. En RFC1 sí se usó la localidad de recogida porque lo que se mide ahí es la demanda de cada pasajero.

El rango se filtra por la fecha de salida porque el ingreso se genera cuando el viaje ocurre, no cuando se publica ni cuando se hace la solicitud.

RFC3 - Consultar viajes por conductor a través del tiempo.

Para esta consulta se interpretó la "evolución" como la cantidad de viajes completados que hizo cada conductor en cada mes, según la fecha de salida. Los filtros de conductor, localidad de origen del viaje y rango de fechas son opcionales, o sea que si se deja uno vacío no restringe el resultado. En APEX se muestra como una gráfica de líneas con una línea por conductor.

```sql
SELECT CONCAT(CONCAT(u.Nombre, ' '), u.Apellido)           AS conductor,
       TO_CHAR(TRUNC(v.Fecha_hora_salida, 'MM'), 'YYYY-MM') AS mes,
       COUNT(*)                                            AS viajes_realizados
FROM Viajes v
INNER JOIN Carros c  ON c.Placa        = v.Carros_Placa
INNER JOIN Usuario u ON u.Documento_ID = c.Usuario_Documento_ID
-- solo viajes realizados (completados)
WHERE EXISTS (SELECT 1
              FROM Estado_viaje ev
              INNER JOIN Estados_viaje esv ON esv.ID_estado = ev.Estados_viaje_ID_estado
              WHERE ev.Viajes_ID_viaje = v.ID_viaje
                AND esv.Nombre_estado = 'Completado')
  -- filtros opcionales
  AND (:P_CONDUCTOR    IS NULL OR u.Documento_ID            = :P_CONDUCTOR)
  AND (:P_LOCALIDAD    IS NULL OR v.Localidades_ID_localidad = :P_LOCALIDAD)
  AND (:P_FECHA_INICIO IS NULL OR v.Fecha_hora_salida >= TO_DATE(:P_FECHA_INICIO, 'YYYY-MM-DD'))
  AND (:P_FECHA_FIN    IS NULL OR v.Fecha_hora_salida <  TO_DATE(:P_FECHA_FIN, 'YYYY-MM-DD') + 1)
GROUP BY u.Documento_ID, u.Nombre, u.Apellido, TRUNC(v.Fecha_hora_salida, 'MM')
ORDER BY TRUNC(v.Fecha_hora_salida, 'MM'), conductor;
```

Tabla	Para qué se usa	Atributos del JOIN
Viajes	Cada fila es un viaje. De aquí sale la fecha y la localidad	Carros_Placa, ID_viaje
Carros	Conecta el viaje con el dueño del vehículo	Placa, Usuario_Documento_ID
Usuario	Identifica al conductor	Documento_ID
Estado_viaje y Estados_viaje	Historial del viaje, dentro de EXISTS	Viajes_ID_viaje, Estados_viaje_ID_estado, ID_estado

En el modelo, el conductor no está guardado en Viajes, sino que se llega a él por el carro que tiene asociado el viaje. Por eso se usó INNER JOIN con Carros y con Usuario: todo viaje tiene un carro y todo carro tiene un dueño, así que no se pierde ningún viaje, y solo aparecen los conductores que sí tienen viajes realizados.

Para revisar que el viaje esté completado se usó EXISTS y no un INNER JOIN, porque un viaje completado tiene tres filas en Estado_viaje (Programado, En curso y Completado), y con un INNER JOIN se contaría tres veces. Con EXISTS solo se revisa que tenga la fila de 'Completado'.

Para agrupar por mes se usó TRUNC(Fecha_hora_salida, 'MM'), que lleva cada fecha al primer día de su mes, incluyendo el año. Si se agrupara por Fecha_hora_salida sin truncar, cada viaje quedaría en su propio grupo. También se agrupa por Documento_ID para que dos conductores con el mismo nombre no se mezclen. El TO_CHAR solo sirve para mostrar el mes bonito, y el orden se hace con la fecha truncada para que quede cronológico.

Para los filtros opcionales se usó el patrón (:P IS NULL OR columna = :P). Si el parámetro está vacío, la condición es verdadera para todas las filas y no filtra nada.

RFA1 - Tablero de control del conductor.

El tablero del conductor muestra cuántos viajes completó, el total de pasajeros que ha transportado, los ingresos que ha obtenido, y el tiempo y la distancia de todos sus viajes. Se tomaron solo los viajes completados, porque el enunciado dice que los tableros se calculan a partir de los viajes completados. Un pasajero transportado es una solicitud que fue aceptada y que no se canceló después.

```sql
SELECT COUNT(DISTINCT v.ID_viaje)          AS "Viajes tomados",
       NVL(SUM(v.Costo), 0)                AS "Total gastado (COP)",
       ROUND(AVG(v.Distancia_estimada), 1)  AS "Distancia promedio (km)",
       ROUND(AVG(v.Duracion_estimada))      AS "Tiempo promedio (min)"
FROM Solicitudes s
INNER JOIN Viajes v ON v.ID_viaje = s.Paradas_viaje_Viajes_ID_viaje
WHERE s.Usuario_Documento_ID = (SELECT u.Documento_ID
                               FROM Usuario u
                               WHERE UPPER(u.Correo) = UPPER(:P_CORREO))
  -- la solicitud fue aceptada...
  AND EXISTS (SELECT 1
              FROM Estados_de_solicitud e
              INNER JOIN Estados_solicitud es
                      ON es.ID_estado_solicitud = e.Estados_solicitud_ID_estado_solicitud
              WHERE e.Solicitudes_ID_solicitud = s.ID_solicitud
                AND es.Nombre_solicitud = 'Aceptada')
  -- ...y no se canceló después
  AND NOT EXISTS (SELECT 1
                  FROM Estados_de_solicitud e
                  INNER JOIN Estados_solicitud es
                          ON es.ID_estado_solicitud = e.Estados_solicitud_ID_estado_solicitud
                  WHERE e.Solicitudes_ID_solicitud = s.ID_solicitud
                    AND es.Nombre_solicitud = 'Cancelada')
  -- el viaje se completó
  AND EXISTS (SELECT 1
              FROM Estado_viaje ev
              INNER JOIN Estados_viaje esv
                      ON esv.ID_estado = ev.Estados_viaje_ID_estado
              WHERE ev.Viajes_ID_viaje = v.ID_viaje
                AND esv.Nombre_estado = 'Completado');

-- Consulta B: zona donde el pasajero más ha pedido viajes
-- (todas sus solicitudes, sin importar el estado)
SELECT z.Nombre_zona AS "Zona más frecuente",
       COUNT(*)      AS "Solicitudes en esa zona"
FROM Solicitudes s
INNER JOIN Paradas p      ON p.ID_parada      = s.Paradas_viaje_Paradas_ID_parada
INNER JOIN Zonas_parada z ON z.ID_zona_parada = p.Zonas_parada_ID_zona_parada
WHERE s.Usuario_Documento_ID = (SELECT u.Documento_ID
                               FROM Usuario u
                               WHERE UPPER(u.Correo) = UPPER(:P_CORREO))
GROUP BY z.ID_zona_parada, z.Nombre_zona
ORDER BY COUNT(*) DESC
FETCH FIRST 1 ROWS ONLY;
```

Tabla	Para qué se usa	Atributos del join
Viajes	Cada fila es un viaje. De aquí sale el costo, la duración y la distancia	Carros_Placa, ID_viaje
Carros	Conecta el viaje con el conductor	Placa, Usuario_Documento_ID
Solicitudes (subconsulta pv)	Cuenta los pasajeros de cada viaje	Paradas_viaje_Viajes_ID_viaje, ID_solicitud
Estados_de_solicitud y Estados_solicitud	Historial de la solicitud, dentro de EXISTS y NOT EXISTS	Solicitudes_ID_solicitud, Estados_solicitud_ID_estado_solicitud, ID_estado_solicitud
Estado_viaje y Estados_viaje	Historial del viaje, dentro de EXISTS	Viajes_ID_viaje, Estados_viaje_ID_estado, ID_estado

Lo más importante de esta consulta es que los indicadores no están al mismo nivel. Los pasajeros y los ingresos se cuentan por solicitud, mientras que el tiempo y la distancia son por viaje. Si se uniera Viajes directo con Solicitudes, la distancia de cada viaje se repetiría una vez por pasajero. Por eso primero se cuentan los pasajeros de cada viaje en una subconsulta (pv), y así cada viaje queda en una sola fila antes de sumar.

Con la subconsulta se usó LEFT JOIN y no INNER JOIN, porque un viaje completado sin pasajeros igual cuenta para el tiempo y la distancia que manejó el conductor. Con INNER JOIN ese viaje desaparecería del resultado. Para esos viajes el conteo de pasajeros queda en NULL, y el NVL lo convierte en 0 para que no dañe las sumas.

Con Carros se usó INNER JOIN porque el conductor no está guardado en Viajes, sino que se llega a él por el carro del viaje, y todo viaje tiene un carro. Los ingresos se calculan como pasajeros por costo en cada viaje, ya que todos los pasajeros pagan el mismo precio. Para los estados se usó EXISTS y NOT EXISTS por la misma razón que en las otras consultas, para no repetir filas por cada estado del historial.

El conductor no se escoge en la página, sino que es el usuario que inició sesión. :APP_USER es la variable de APEX con el nombre de usuario de la sesión, que en esta aplicación es el correo, entonces con una subconsulta sobre Usuario se busca el documento que corresponde a ese correo. El UPPER en los dos lados evita problemas si el correo se escribió con mayúsculas al iniciar sesión. Así cada conductor ve solo su propio tablero.

RFA2 - Tablero de control del pasajero.

El tablero del pasajero muestra cuántos viajes ha tomado, cuánto ha gastado en total, la distancia y el tiempo promedio de sus viajes, y la zona en la que más ha pedido viajes. Se tomó como "viaje tomado" una solicitud del pasajero que fue aceptada, que no se canceló después y cuyo viaje se completó, porque el enunciado dice que los tableros se calculan a partir de los viajes completados.

Este tablero se partió en dos consultas. Las métricas de viajes, gasto y promedios resumen todo en una sola fila, mientras que la zona más frecuente necesita agrupar por zona. Si se juntan en una misma consulta, los números se dañan, entonces cada una va en su propia región de APEX.

Consulta A: métricas del pasajero.

```sql
SELECT COUNT(DISTINCT v.ID_viaje)          AS "Viajes tomados",
       NVL(SUM(v.Costo), 0)                AS "Total gastado (COP)",
       ROUND(AVG(v.Distancia_estimada), 1) AS "Distancia promedio (km)",
       ROUND(AVG(v.Duracion_estimada))     AS "Tiempo promedio (min)"
FROM Solicitudes s
INNER JOIN Viajes v ON v.ID_viaje = s.Paradas_viaje_Viajes_ID_viaje
WHERE s.Usuario_Documento_ID = (SELECT u.Documento_ID
                               FROM Usuario u
                               WHERE UPPER(u.Correo) = UPPER(:P_CORREO))
  -- la solicitud fue aceptada...
  AND EXISTS (SELECT 1
              FROM Estados_de_solicitud e
              INNER JOIN Estados_solicitud es
                      ON es.ID_estado_solicitud = e.Estados_solicitud_ID_estado_solicitud
              WHERE e.Solicitudes_ID_solicitud = s.ID_solicitud
                AND es.Nombre_solicitud = 'Aceptada')
  -- ...y no se canceló después
  AND NOT EXISTS (SELECT 1
                  FROM Estados_de_solicitud e
                  INNER JOIN Estados_solicitud es
                          ON es.ID_estado_solicitud = e.Estados_solicitud_ID_estado_solicitud
                  WHERE e.Solicitudes_ID_solicitud = s.ID_solicitud
                    AND es.Nombre_solicitud = 'Cancelada')
  -- el viaje se completó
  AND EXISTS (SELECT 1
              FROM Estado_viaje ev
              INNER JOIN Estados_viaje esv
                      ON esv.ID_estado = ev.Estados_viaje_ID_estado
              WHERE ev.Viajes_ID_viaje = v.ID_viaje
                AND esv.Nombre_estado = 'Completado');
```

Tabla	Para qué se usa	Atributos del JOIN
Solicitudes	Conecta al pasajero con los viajes que pidió	Paradas_viaje_Viajes_ID_viaje, Usuario_Documento_ID, ID_solicitud
Viajes	Da el costo, la distancia y la duración	ID_viaje
Estados_de_solicitud y Estados_solicitud	Historial de la solicitud, dentro de EXISTS y NOT EXISTS	Solicitudes_ID_solicitud, Estados_solicitud_ID_estado_solicitud, ID_estado_solicitud
Estado_viaje y Estados_viaje	Historial del viaje, dentro de EXISTS	Viajes_ID_viaje, Estados_viaje_ID_estado, ID_estado

Se parte de Solicitudes porque es la tabla que relaciona al pasajero con los viajes. Con Viajes se usó INNER JOIN porque toda solicitud pertenece a un viaje. No se unió con Usuario porque la solicitud ya guarda el documento del pasajero, entonces basta con filtrar por Usuario_Documento_ID. Ese documento

se obtiene del usuario que inició sesión: :APP_USER tiene el correo de la sesión, y con una subconsulta sobre Usuario se busca el documento que le corresponde.

Para los estados se usó EXISTS y NOT EXISTS por la misma razón que en RFC2: así no se repiten filas por cada estado del historial. Después de estos filtros, cada fila que queda es un viaje que el pasajero sí hizo.

Para contar los viajes se usó COUNT(DISTINCT v.ID_viaje), por si un pasajero tuviera dos solicitudes aceptadas en el mismo viaje. El gasto total es SUM(v.Costo), porque cada fila es un cupo que pagó. El NVL hace que se muestre 0 en vez de vacío cuando el pasajero todavía no tiene viajes, y el ROUND deja los promedios con un número de decimales que se pueda leer en el tablero.

Consulta B: zona donde más ha pedido viajes.

```
SELECT z.Nombre_zona AS "Zona más frecuente",
       COUNT(*)      AS "Solicitudes en esa zona"
FROM Solicitudes s
INNER JOIN Paradas p      ON p.ID_parada      = s.Paradas_viaje_Paradas_ID_parada
INNER JOIN Zonas_parada z ON z.ID_zona_parada = p.Zonas_parada_ID_zona_parada
WHERE s.Usuario_Documento_ID = (SELECT u.Documento_ID
                               FROM Usuario u
                               WHERE UPPER(u.Correo) = UPPER(:P_CORREO))
GROUP BY z.ID_zona_parada, z.Nombre_zona
ORDER BY COUNT(*) DESC
FETCH FIRST 1 ROWS ONLY;
```

Tabla	Para qué se usa	Atributos del join
Solicitudes	Cada fila es una solicitud del pasajero, con su punto de recogida	Paradas_viaje_Paradas_ID_parada, Usuario_Documento_ID
Paradas	Ubica el punto de recogida en una zona	ID_parada, Zonas_parada_ID_zona_parada
Zonas_parada	Da el nombre de la zona por la que se agrupa	ID_zona_parada

La zona sale del punto de recogida que eligió el pasajero, por eso se llega por Solicitudes → Paradas → Zonas_parada. Se usó INNER JOIN porque toda solicitud tiene una parada y toda parada tiene una zona, ya que las FK son obligatorias.

Aquí no se filtra por estado porque el enunciado habla de la zona donde más ha "pedido" viajes, y pedir incluye también las solicitudes que fueron rechazadas. Se agrupa por el ID de la zona además del nombre, para que dos zonas con el mismo nombre no se mezclen. Al final, se ordena de mayor a menor y con FETCH FIRST 1 ROWS ONLY se deja solo la primera, que es la zona más frecuente.

Datos de consulta:

Credenciales de conductores con al menos 1 viaje completado:

usuario16@uniandes.edu.co
usuario17@uniandes.edu.co
usuario18@uniandes.edu.co
usuario19@uniandes.edu.co
usuario20@uniandes.edu.co

Clave: clave123

6. Evidencias de ejecución.

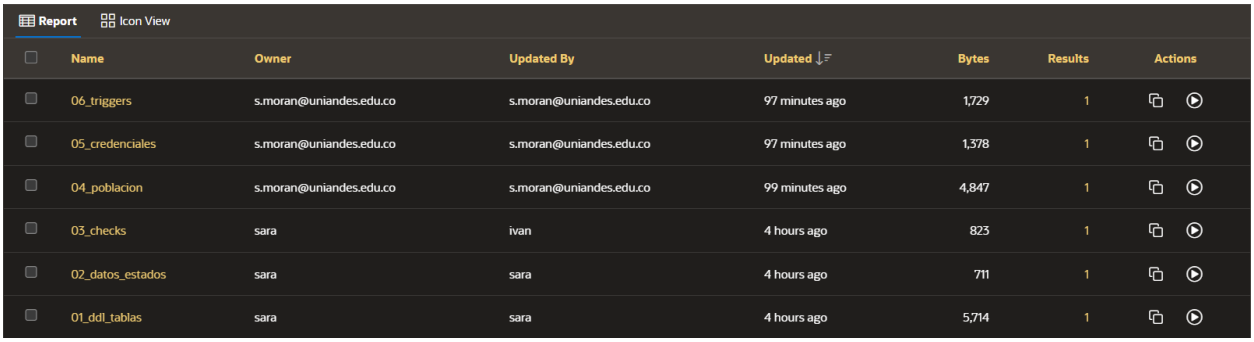


Ilustración 2.Scripts creados en SQL Workshop > SQL Scripts